HiddenGem

Database Design -- Revised

Singular table names, tableNameID primary key convention & corrected foreign keys

Prepared by: Cheryl Wang Smith
Course: SWDV 691: SOFTWARE DEVELOPMENT CAPSTONE 1W 26/SP2
Date: March 29, 2026

What Changed From the Original Database Design

Following standard database practice, all table names are singular and all primary keys follow the tableNameID convention.

- Singular table names: user, gem, photo, tag, comment, save, follow
- Primary keys follow the pattern tableNameID: userID, gemID, photoID, tagID, commentID
- Composite primary key tables (save, follow) use the referenced tableNameIDs as their composite PK columns
- Foreign keys use the same tableNameID naming as the primary key they reference: gem.userID references user.userID

Section 1: Database Technology Choice

1.1 Choice: PostgreSQL (Relational Database)

HiddenGem will use PostgreSQL as its primary data store, hosted on Railway alongside the Node.js/Express API. The decision is grounded in the specific data access patterns the application requires.

1.2 Justification

HiddenGem's data is highly relational by nature. Every core feature involves joins across multiple entities:

- The Discover feed must join gem with user (to show submitter names and avatars), photo (to show cover images), tag (to support filtering), and save (to show save counts).
- The follow graph requires a self-referential many-to-many join on the user table -- a query pattern that relational databases handle cleanly with a simple join table.
- The Community page must determine, for each listed user, whether the current user is already following them -- a point-lookup on the follow join table using followerID and followingID.
- Privacy filtering (Public / Followers Only / Private) requires the API to join gem against follow at query time to determine visibility, naturally expressed in SQL with a WHERE clause and subquery.
- Comment author badge logic compares comment.userID against gem.userID at query time -- a simple equijoin.

1.3 Database User Accounts

Two database-level users are created in PostgreSQL. This separation limits the blast radius if either set of credentials is compromised.

DB user	Permissions	Used by
`hiddengem_app`	SELECT, INSERT, UPDATE, DELETE on all application tables	Node.js/Express API at runtime. Used for all read/write operations during normal application use.
`hiddengem_admin`	ALL PRIVILEGES on hiddengem database	Migrations, schema changes, and administrative tasks only. Never used by the running application.

Section 2: Entity Relationships

The following table summarizes all relationships between tables using the revised singular naming and tableNameID key convention. The ERD diagram is provided separately as a visual companion.

From table	To table	Type	Via / key
`user`	`gem`	**One-to-many**	gem.userID → user.userID -- one user submits many gems
`user`	`comment`	**One-to-many**	comment.userID → user.userID -- one user writes many comments
`user`	`save`	**One-to-many**	save.userID → user.userID -- one user saves many gems
`user`	`follow`	**Many-to-many**	follow.followerID → user.userID AND follow.followingID → user.userID -- self-referential
`gem`	`photo`	**One-to-many**	photo.gemID → gem.gemID -- one gem has up to 3 photos
`gem`	`tag`	**One-to-many**	tag.gemID → gem.gemID -- one gem has many tags (stored as rows)
`gem`	`comment`	**One-to-many**	comment.gemID → gem.gemID -- one gem receives many comments
`gem`	`save`	**Many-to-many**	save join table: user ↔ gem bidirectional via userID + gemID composite PK

Section 3: Table Specifications & Application Context

3.1 user

The user table is the root of the entire data model. Every other table references it directly or indirectly. It stores identity, authentication credentials, and public profile information.

user			
Column	**Type**	**Constraint**	**Description**
`userID`	`UUID`	PK	Primary key, generated by the database (gen_random_uuid()).
`username`	`VARCHAR(40)`	UNIQUE NOT NULL	URL-safe @handle shown on profiles and gem cards. Used in /profile/:username routing.
`display_name`	`VARCHAR(80)`	NOT NULL	Full name shown on gem detail pages and comment threads.
`email`	`VARCHAR(255)`	UNIQUE NOT NULL	Login credential. Unique constraint prevents duplicate accounts.
`password_hash`	`VARCHAR(72)`	NOT NULL	Bcrypt hash of the user's password. Plaintext is never stored.
`bio`	`TEXT`	NULLABLE	Optional self-description shown on the profile page left panel.
`avatar_url`	`VARCHAR(500)`	NULLABLE	Cloudinary CDN URL for the profile photo. NULL means the UI renders an initials badge instead.
`is_verified`	`BOOLEAN`	DEFAULT FALSE	Set to TRUE when the user clicks the email verification link. Unverified users cannot submit gems.
`created_at`	`TIMESTAMP`	DEFAULT NOW()	Shown on Community page user cards as "Member Since".

Application context: when Maya signs up, a row is inserted with is_verified = FALSE. After she clicks the verification link, the API issues an UPDATE to set is_verified = TRUE. When James views Maya's profile at /profile/mayachen, the API queries this table by username. The follower/following counts displayed are computed by counting rows in the follow table, not stored here.

3.2 gem

The gem table is the central content table of the application. Each row represents one location that a user has personally discovered and submitted. It anchors the map, the discovery feed, and the detail page.

gem

Column	Type	Constraint	Description
gemID	UUID	PK	Primary key.
userID	UUID	FK → user.userID	The submitter. Used to display author info on cards and detail pages, and to enforce edit/delete authorization.
name	VARCHAR(120)	NOT NULL	The title shown on the gem card and detail page.
description	TEXT	NOT NULL	Full description written by the submitter. Supports #hashtag and @mention inline text.
category	VARCHAR(40)	NOT NULL	Primary category enum: Nature, Food, Art, Architecture, Historic, Other. Used for map pin color-coding and discovery filters.
latitude	DECIMAL(9,6)	NOT NULL	Geographic latitude. Used to render the pin on Google Maps and compute distance.
longitude	DECIMAL(9,6)	NOT NULL	Geographic longitude. Paired with latitude for geocoding and radius search.
location_label	VARCHAR(200)	NULLABLE	Human-readable address string. Shown below the map on the detail page.
privacy	VARCHAR(20)	DEFAULT 'public'	Visibility enum: public, followers_only, private. The API filters by this on every query.
view_count	INTEGER	DEFAULT 0	Incremented on each unique visit to the gem detail page. Shown in Edit Post statistics panel.
is_flagged	BOOLEAN	DEFAULT FALSE	Set TRUE when reported. Flagged gems excluded from all public queries pending review.
created_at	TIMESTAMP	DEFAULT NOW()	Submission time. Used to sort the discovery feed by "newest".

Application context: when James submits a gem, the API inserts a row with his userID, coordinates from the location picker, and privacy = 'public'. The Discover feed query joins gem with user (on gem.userID = user.userID) and aggregates save counts from the save table. Sofia's distance filter appends a WHERE clause computing Haversine distance against latitude/longitude.

3.3 photo

The photo table stores references to images that users attach to their gems. Actual image files are stored in Cloudinary; this table persists the CDN URLs and controls display order. Separating photos into their own table allows the API to add, remove, or reorder photos independently without rewriting the entire gem row.

photo			
Column	**Type**	**Constraint**	**Description**

photoID	UUID	PK	Primary key.
gemID	UUID	FK → gem.gemID	The gem this photo belongs to. Deleting a gem cascades to delete all associated photo rows.
url	VARCHAR(500)	NOT NULL	Cloudinary CDN URL returned after a successful upload.
display_order	INTEGER	NOT NULL DEFAULT 0	0 = the cover image shown in feed cards. Higher = additional photos in the detail page carousel. Maximum 3 photos per gem.

Application context: when Maya submits a gem with 2 photos, the API uploads both to Cloudinary and inserts 2 rows with display_order 0 and 1. The feed card queries WHERE gemID = ? AND display_order = 0 to retrieve only the cover image. The full carousel is fetched only when the user opens the detail page.

3.4 tag

The tag table stores the hashtag taxonomy users attach to their gems (e.g. #waterfall, #hiking, #nature). Tags are stored as individual rows rather than a comma-delimited string, allowing the Discover page to execute efficient tag-filter queries using a JOIN rather than a LIKE search on a string column.

tag			
Column	**Type**	**Constraint**	**Description**
tagID	UUID	PK	Primary key.
gemID	UUID	FK → gem.gemID	The gem this tag belongs to. Deleting a gem cascades to delete all its tags.
name	VARCHAR(60)	NOT NULL	Tag value without the # prefix, normalized to lowercase (e.g. "waterfall"). A GIN index on this column accelerates the discovery tag filter.

Application context: when Sofia clicks the #nature tag filter pill in the sidebar, the API appends JOIN tag t ON t.gemID = g.gemID WHERE t.name = 'nature' to the main gem query. On the user profile page, the tag pills above the gem grid are populated by SELECT DISTINCT t.name FROM tag t JOIN gem g ON t.gemID = g.gemID WHERE g.userID = ?.

3.5 comment

The comment table stores the community comment threads that appear on each gem detail page. Comments allow explorers to ask questions about access and conditions, and allow the original submitter to respond. The table is unbounded -- a gem can accumulate any number of comments over time.

comment			
Column	**Type**	**Constraint**	**Description**
commentID	UUID	PK	Primary key.
gemID	UUID	FK → gem.gemID	The gem this comment belongs to. Deleting a gem cascades to delete all comments.
userID	UUID	FK → user.userID	The commenter. Used to display their avatar and @handle, and to enforce delete authorization.
body	TEXT	NOT NULL	The comment text. Rendered in the comment thread on the gem detail page.
created_at	TIMESTAMP	DEFAULT NOW()	Used to order comments chronologically (ASC). Oldest comment appears first.

Application context: the comment thread query is SELECT c.*, u.display_name, u.avatar_url, u.username, (c.userID = g.userID) AS is_author FROM comment c JOIN user u ON c.userID = u.userID JOIN gem g ON c.gemID = g.gemID WHERE c.gemID = ? ORDER BY c.created_at ASC. The is_author boolean drives the "Author" badge in the UI.

3.6 save

The save table implements the bookmark feature -- users can save any gem to their personal list to revisit later. It is a pure join table implementing a many-to-many relationship between user and gem. The composite primary key (userID + gemID) ensures a user can only save a given gem once, and makes the "is this gem already saved?" lookup a fast primary-key point read.

save			
Column	**Type**	**Constraint**	**Description**
userID	UUID	PK, FK → user.userID	The user who saved the gem. Half of the composite PK.
gemID	UUID	PK, FK → gem.gemID	The gem that was saved. Half of the composite PK.
saved_at	TIMESTAMP	DEFAULT NOW()	When the user saved the gem. Used to sort the Saved tab on the profile by most recently saved.

Application context: the save count on each gem card is computed by SELECT COUNT(*) FROM save WHERE gemID = ? -- a fast index scan on gemID. When Maya taps the bookmark icon, the API inserts a row. If she taps again, it issues a DELETE. The Saved tab on her profile is populated by: SELECT g.* FROM gem g JOIN save s ON s.gemID = g.gemID WHERE s.userID = ? ORDER BY s.saved_at DESC.

3.7 follow (Stretch Feature)

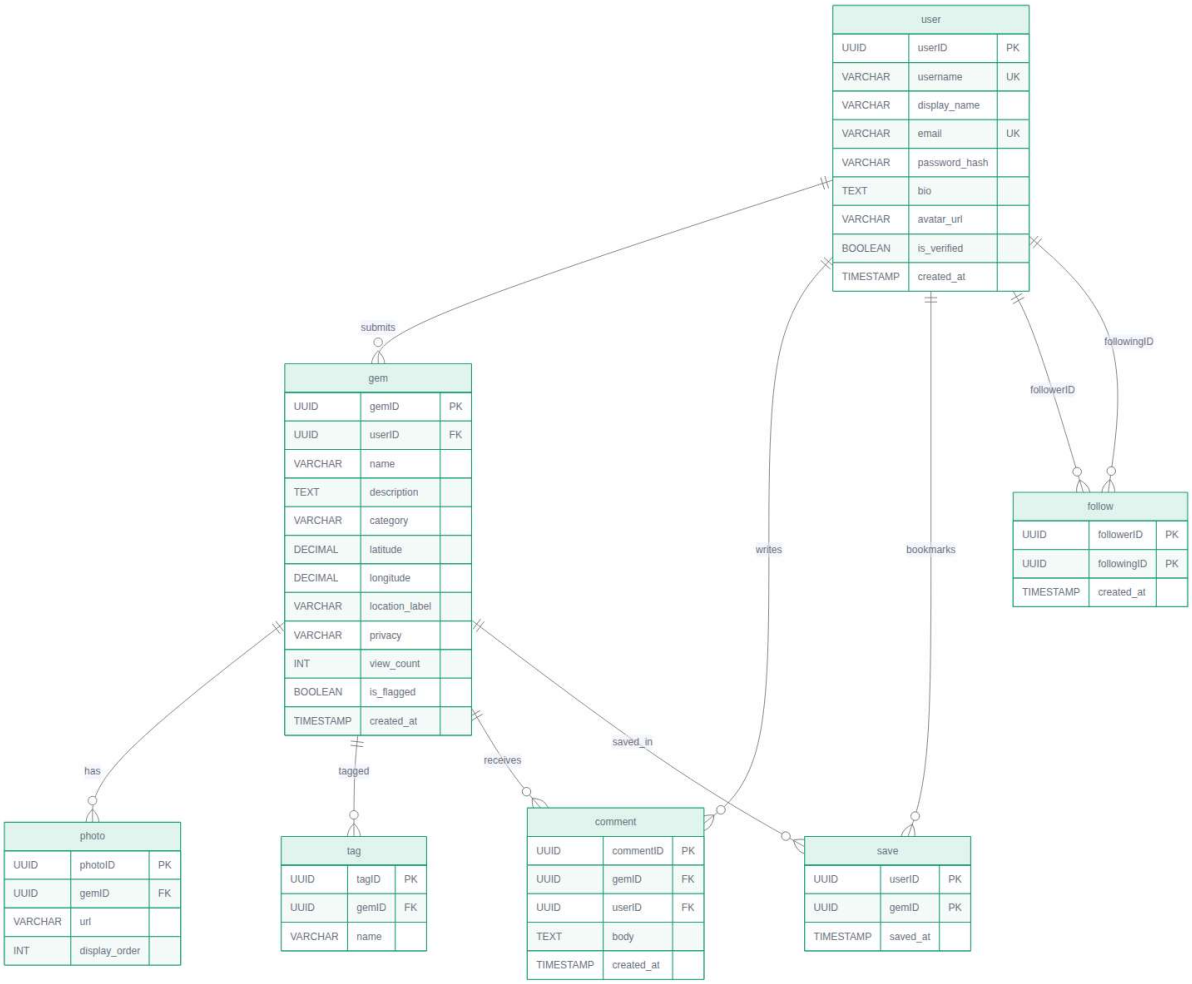
The follow table implements the social graph that is central to HiddenGem's personalized discovery experience. It is a self-referential many-to-many join on the user table: any user can

follow any other user. The composite primary key (followerID + followingID) enforces uniqueness and makes point-lookups fast. This table transforms HiddenGem from a static map into a living, personalized feed.

follow

Column	Type	Constraint	Description
followerID	UUID	PK, FK → user.userID	The user who is doing the following. Half of the composite PK.
followingID	UUID	PK, FK → user.userID	The user being followed. Half of the composite PK.
created_at	TIMESTAMP	DEFAULT NOW()	When the follow relationship was established.

Application context: when Sofia clicks "Follow" on James's card, the API inserts (followerID = Sofia.userID, followingID = James.userID). The "Following" button state is determined by: SELECT 1 FROM follow WHERE followerID = ? AND followingID = ? LIMIT 1 -- a primary-key point read. The follower count on a profile is: SELECT COUNT(*) FROM follow WHERE followingID = ?. Sofia's personalized Discover feed is: SELECT g.* FROM gem g WHERE g.userID IN (SELECT followingID FROM follow WHERE followerID = ?) AND g.privacy != 'private' ORDER BY g.created_at DESC.


user
UUID userID PK
VARCHAR username UK
VARCHAR display_name
VARCHAR email UK
VARCHAR password_hash
TEXT bio
VARCHAR avatar_url
BOOLEAN is_verified
TIMESTAMP created_at
submits
gem
UUID gemID PK
UUID userID FK
VARCHAR name
TEXT description
VARCHAR category
DECIMAL latitude
DECIMAL longitude
VARCHAR location_label
VARCHAR privacy
INT view_count
BOOLEAN is_flagged
TIMESTAMP created_at
followingID
followerID
follow
UUID followerID PK
UUID followingID PK
TIMESTAMP created_at
writes
bookmarks
has
tagged
receives
saved_in
photo
UUID photoID PK
UUID gemID FK
VARCHAR url
INT display_order
tag
UUID tagID PK
UUID gemID FK
VARCHAR name
comment
UUID commentID PK
UUID gemID FK
UUID userID FK
TEXT body
TIMESTAMP created_at
save
UUID userID PK
UUID gemID PK
TIMESTAMP saved_at

End